

Mobile (Flutter) Coding Standard — v2.0

Owner: Shanmugarajeshwaran · **Approved:** Praveen (CTO)

Stack: Flutter · Dart · BLoC · Android and iOS

Supersedes: Mobile (Flutter/Dart) Coding Standard v1.0

Scope and status

This standard applies to **new and changed code**. Existing code is not retrofitted — a rule arriving here does not create a backlog of rewrites. When you touch a file, the code you write in it follows this document.

This document is self-contained. Every rule is stated here. v1.0 referred to project-level documents for security, logging, release and workflow rules; those references are replaced with the rules themselves, so a developer can act on this document without finding four others first.

Sections 1–13 mirror the Angular, Next.js, Node, Python and Spring Boot standards section for section. Section 14 is mobile-specific.

Version policy

Projects run a supported stable Flutter release, upgraded at least once per release cycle as planned work. The Flutter SDK version is pinned in the repo so every developer and the CI runner build with the same toolchain.

Enforcement tags

Tag	Meaning
[LINTER]	<code>dart analyze</code> with our <code>analysis_options.yaml</code> , or the compiler.
[CI]	A pipeline step or Git-host setting — Gitleaks, dependency audit, coverage gate, branch protection.
[NEITHER]	A human must judge it in review.
[LINTER + NEITHER]	The tool covers part; review covers the rest.

Check: notes name the specific lint rule, or explain why a rule is not automatable.

1. Naming

- [LINTER] Files and folders `snake_case`. Classes and widgets `UpperCamelCase`. Variables, methods **and constants** `lowerCamelCase`. **Check:** `file_names`, `camel_case_types`, `constant_identifier_names`.

Constants are `lowerCamelCase` here, not `UPPER_SNAKE_CASE` as in every other stack. That is the Dart convention and fighting it costs more than it gains. It is the one naming divergence in the company and it is deliberate.

- [LINTER] Private members start with `_`. Anything not used outside its library is private. **Check:** `library_private_types_in_public_api`.
- [NEITHER] Names carry units where ambiguity is possible: `timeoutSeconds`, `amountMinor`, `distanceMetres`. *Reason:* nobody should have to read the implementation to know whether a number is seconds or milliseconds.
- [NEITHER] Booleans read as assertions — `isActive`, `hasExpired`, `canSubmit`. No `data`, `info`, `temp`, `flag` as a name; if that is the best name available, the variable is doing too much.
- [NEITHER] Functions returning a nullable result start with `find`; functions that throw when missing start with `get`.
- [NEITHER] Widget files are named after the widget they export: `loan_card.dart` exports `LoanCard`.

1.1 Wire format across stacks

[NEITHER] **JSON payloads are camelCase on the wire, in both directions.** Dart is already camelCase, so nothing converts here.

If an endpoint returns `snake_case`, that is a backend defect — raise it rather than writing a mapping layer in the app. A per-feature mapping layer is how one inconsistent endpoint becomes twelve, and a mobile app cannot be hot-fixed when the mapping is wrong.

[NEITHER] Models are generated or hand-written with explicit `fromJson`, never `dynamic` passed through the app. An unexpected `null` should fail at the boundary with a clear message, not three screens deep.

2. Project structure

Feature-first.

```
lib/
  main.dart
  app/                # MaterialApp, routing, theme wiring, DI setup
  features/
    login/
      presentation/   # screens, widgets, bloc
      domain/         # business rules and contracts
      data/           # API and storage access
  core/
    network/          # api client, interceptors, error mapping
    storage/          # secure storage, preferences
    observability/    # logger, correlation id, alerting
    theme/            # tokens, ThemeExtension
    errors/           # AppException, failure types
  test/features/login/ # tests mirror the source tree
  android/ ios/       # native settings
```

- **[NEITHER]** Feature-first on every new project. No exceptions. Existing projects on another layout are not migrated.
- **[NEITHER]** Layer a feature when the feature earns it. A screen that reads one endpoint and shows a list does not need a `domain/` layer with an abstract repository and a use-case class per action — that is ceremony. Add the layer when there is real business logic to isolate, a second data source, or logic shared across features.
- **[NEITHER]** Presentation never calls the API directly. Data access goes through the feature's `data/` layer, even when that layer is thin.
- **[NEITHER]** No cross-feature imports. Features talk through `core/`, never to another feature's internals.

3. Error handling & async safety

3.1 Typed errors, mapped once

- **[NEITHER]** The network layer throws typed errors. Screens never see a raw `DioException`, `SocketException` or a status code.
- **[NEITHER]** Users never see a raw error object, stack trace or backend message. Map to a human message; keep the technical detail in the log.
- **[NEITHER]** Every screen that loads data handles **four states: loading, empty, error, success**. A screen that only handles the happy path is not done.

```
// core/errors/app_exception.dart
class AppException implements Exception {
  const AppException(this.code, this.message, {this.correlationId, this.details});

  final String code;           // machine-readable: 'LOAN_ALREADY_SETTLED'
  final String message;        // developer-facing; never shown to a user
  final String? correlationId;
  final Map<String, List<String>>? details;
}

const _messages = <String, String>{
  'NETWORK_UNAVAILABLE': 'We could not reach the server. Check your connection and try again.',
  'SESSION_EXPIRED': 'Your session has ended. Please sign in again.',
  'LOAN_ALREADY_SETTLED': 'This loan has already been settled.',
};

String userMessage(Object error) => error is AppException
  ? _messages[error.code] ?? 'Something went wrong. Please try again.'
  : 'Something went wrong. Please try again.';
```

Branch on `code`, never on `message`. Message text changes; codes are a contract.

[NEITHER] Show the correlation id on the error state. It costs nothing and turns a support ticket into a log lookup.

3.2 Async safety — the rules that prevent crashes

These are the most important rules in this document. Mobile users leave screens, lose signal and tap twice.

- [LINTER] Check `mounted` / `context.mounted` after every `await` before touching a widget or context. Reason: the user may have left the screen while the request was in flight; using a disposed context throws. Check: `use_build_context_synchronously`.

```
// ✖ crashes if the user navigates away during the request
Future<void> _submit() async {
  final result = await repository.settle(loanId);
  Navigator.of(context).pop(result);
}

// ✔
Future<void> _submit() async {
  final result = await repository.settle(loanId);
  if (!context.mounted) return;
  Navigator.of(context).pop(result);
}
```

- [NEITHER] In a BLoC, check `isClosed` before emitting after an `await`, and `emit.isDone` in a cancellable or completed handler. Emitting after close throws and takes the screen with it.

```
Future<void> _onSettle(SettleLoan event, Emitter<LoanState> emit) async {
  emit(const LoanState.loading());
  try {
    final loan = await _repository.settle(event.id);
    if (isClosed || emit.isDone) return;
    emit(LoanState.ready(loan));
  } on AppException catch (e) {
    if (isClosed || emit.isDone) return;
    emit(LoanState.error(userMessage(e), e.correlationId));
  }
}
```

- [NEITHER] Cancel work you own when it is no longer needed. Every `StreamSubscription`, `Timer`, `AnimationController` and `TextEditingController` created by a widget is disposed in `dispose()`.
- [NEITHER] Prevent double submission. Disable the control while a request is in flight. Users tap twice when the network is slow, and a second tap on a payment is not a harmless retry.
- [NEITHER] Every network call has a timeout. A call without one hangs until the user force-quits.
- [LINTER] No empty `catch`, and no `catch` that only logs and continues. Check: `empty_catches`.
- [NEITHER] Catch errors only to handle or report them. Keep the stack trace — `catch (e, stackTrace)` and pass both on.

4. Logging, observability & alerting

4.1 Log levels

Level	Meaning
error	A human needs to look at this.
warning	Degraded but handled.
info	A state change worth auditing.
debug	Off in release builds.

4.2 Redaction enforced in code

[NEITHER] **Never logged, in any environment:** tokens, passwords, OTPs, API keys, wallet private keys, seed phrases, `Authorization` headers, card or bank data, full request or response bodies containing personal data, whole state objects.

A denylist in a document is a hope. The logger applies it:

```
// core/observability/redact.dart
const _deny = {
  'authorization', 'password', 'passwd', 'secret', 'token', 'access_token',
  'refresh_token', 'api_key', 'api_key', 'private_key', 'mnemonic', 'seed',
  'otp', 'pin', 'cvv', 'card_number', 'aadhaar', 'pan', 'ssn', 'email', 'phone',
};
const _redacted = '[REDACTED]';
const _maxDepth = 6;

Object? redact(Object? value, [int depth = 0]) {
  if (depth > _maxDepth) return '[TRUNCATED]';
  if (value is Map) {
    return value.map((k, v) => MapEntry(
      k,
      _deny.contains(k.toString().toLowerCase()) ? _redacted : redact(v, depth + 1),
    ));
  }
  if (value is List) return value.map((v) => redact(v, depth + 1)).toList();
  return value;
}
```

- [LINTER] **No `print()`, `debugPrint()` or `dart:developer` logging in app code.** Use the logger, so levels, redaction and reporting apply.
Check: `avoid_print`, plus a `no-restricted` rule on `debugPrint`.

If something sensitive does reach a log or a crash report, treat it as a leaked credential and rotate it. It has already been shipped and retained. Removing the line changes nothing.

4.3 Crash and error reporting

[NEITHER] Every build reports crashes and unhandled errors to Sentry with app version, build number, environment, screen, user id (**never** name, email or phone) and correlation id.

```
// main.dart
Future<void> main() async {
  await SentryFlutter.init(
    (options) {
      options.dsn = env.sentryDsn;
      options.environment = env.environment;
      options.release = '${packageInfo.version}+${packageInfo.buildNumber}';
      options.sendDefaultPii = false;
      options.tracesSampleRate = env.isProduction ? 0.1 : 1.0;
      options.beforeSend = (event, hint) {
        // Second line of defence behind redact().
        event.request?.headers.removeWhere((k, _) => _deny.contains(k.toLowerCase()));
        return event..extra = redact(event.extra) as Map<String, dynamic>;
      };
    },
    appRunner: () => runApp(const App()),
  );
}
```

- [NEITHER] **`sendDefaultPii` is false.** Clear the user on logout.
- [CI] **Debug symbols are uploaded on every release build.** A crash report from an obfuscated build without symbols is unreadable, and that is the build your users are running.
- [NEITHER] **Someone is assigned to watch the crash dashboard.** A crash report nobody reads is a log file by another name.

4.4 Correlation IDs

[NEITHER] Every request sends `X-Correlation-Id`. The app generates it; our backends log it and return it. Include it in error reports and show it on the error screen.

5. Testing expectations

The developer proves the code works before it reaches QA.

- [NEITHER] **Unit tests required for:** money and amount calculations, date and schedule logic, state-transition rules, BLoC event-to-state mappings, and any pure function with a branch.
- [NEITHER] **Widget tests for changed interactions** — what the user sees and what happens on tap. Not internal state. A snapshot-only test is not a test.
- [NEITHER] **Integration or on-device checks for critical and native flows** — payments, auth, camera, notifications, deep links. These break on a real device in ways no unit test sees.
- [NEITHER] Record existing behaviour with tests before restructuring old code; verify it afterwards.
- [NEITHER] **Control time, network and test data.** A test that depends on `DateTime.now()` or a live API fails randomly, then gets ignored, then gets deleted.
- [NEITHER] Every fixed bug ships with a test that fails without the fix.
- [NEITHER] **Choose tests by risk, not a fixed coverage percentage.** Formatting-only edits need no new test.
- [CI] All required tests pass before merge. A developer does not skip or delete a failing test to get a PR through.

6. PR & merge rules

- [CI] **One approval minimum, from someone other than the author, on every PR in every project.** Two approvals when the change touches auth, payments, release configuration or `core/`. **Check:** branch protection plus CODEOWNERS.
- [NEITHER] **One purpose per PR. Target 400 handwritten lines added or deleted**, including tests but excluding generated and lock files. Larger PRs are agreed with the reviewer in advance and carry a note on how to review them.
- [CI] Blocks merge: failing CI, analyzer errors, self-approval, protected-branch bypass, unresolved blocking comments.
- [CI + NEITHER] **Required PR description fields:** what changed · why · how it was tested · security/architecture impact · breaking changes · related ticket · release requirements. Write “None” where a field does not apply — **silence is not the same as nothing**. *“Tested locally” is not an answer for how it was tested.*
- [NEITHER] **Mark review comments Blocker or Suggestion** and say why. A reviewer who does not distinguish them leaves the author guessing which comments must be addressed.
- [NEITHER] **AI-written code is the author’s responsibility.** The person raising the PR understands every line in it and can explain it in review. An AI review does not replace a human review, and an unverified AI result is never presented as a tested one.
- [CI] Gitleaks and a dependency audit run on every PR. A detected secret blocks the merge **and triggers rotation** — once committed it is in the history.

On the review requirement. Roughly half the rules in this document are [NEITHER]. On a project where review is switched off, half this standard does not exist — including every async-safety and security rule. Exceptions are granted by the CTO for a named person, not chosen per project.

7. Review checklist

Mark an item N/A with a short reason where it does not apply.

1. [NEITHER] Does the change match the ticket, and nothing more?
2. [NEITHER] `mounted / context.mounted` checked after every await? (3.2)
3. [NEITHER] `isClosed / emit.isDone` checked before emitting after an await? (3.2)
4. [NEITHER] Controllers, subscriptions and timers disposed?
5. [NEITHER] Double submission prevented on anything that writes?
6. [NEITHER] Every network call has a timeout?
7. [NEITHER] Loading, empty, error and success states all handled?
8. [NEITHER] **Money handled per section 12 — no double anywhere near an amount?**
9. [LINTER + NEITHER] Any secret, token or PII in code, logs, crash reports or test fixtures?

10. [NEITHER] Sensitive data in secure storage, not `SharedPreferences` ? (8.2)
11. [NEITHER] Any work started or state changed inside `build()` ? (10)
12. [LINTER] `const` constructors used where possible?
13. [NEITHER] Raw colour, spacing or text-style value instead of a theme token? (13.2)
14. [NEITHER] Both Android and iOS checked for the affected flow?
15. [NEITHER] Offline and poor-connectivity behaviour considered? (14.1)
16. [NEITHER] Tests added for the logic changed and for the bug being fixed?
17. [NEITHER] Package, permission or release changes explained in the description?

8. Security

8.1 The app binary is public

[NEITHER] Anything shipped in the app is readable. `--dart-define`, `.env` files bundled as assets, and obfuscation do **not** hide a value. An installed APK or IPA can be unpacked and the strings read in minutes.

So: **no server secret, API key, signing key or private key is ever in the app**. When a feature needs a third-party service that requires a key, the app calls our backend and our backend calls the third party.

```
✗ App —key→ ThirdParty
✓ App —→ Our API —key from AWS Secrets Manager→ ThirdParty
```

If a vendor's documentation shows a mobile-side key, that key is either publishable by design or the documentation is showing a prototype. **If you are unsure which you are holding, you are holding a secret.**

[CI] Never commit real credentials, secret-filled configuration, keystores or signing keys. **Check:** Gitleaks, blocking.

8.2 Credentials on the device

- [NEITHER] Session credentials live in platform-backed secure storage — `flutter_secure_storage`, which uses Keychain on iOS and Keystore on Android. **Never** `SharedPreferences`, which is a plain file readable on a rooted or jailbroken device and included in some backup mechanisms.
- [NEITHER] Credentials are cleared on logout, including any cached user object, and the crash-reporting user is cleared with them.
- [NEITHER] Everything issued for verification expires — sessions, tokens, OTPs, reset links. The app handles expiry by refreshing once and then signing the user out; it does not loop.
- [NEITHER] **One refresh at a time**. When several requests fail with 401 together, exactly one refresh runs and the rest wait for it. *Reason:* refresh tokens rotate on use, so parallel refreshes invalidate each other and the user is signed out at random.

8.3 Server-side authorisation

[NEITHER] Hiding a button is UX, not security. Every protected action is authorised server-side.

A mobile app is fully under the user's control — it can be patched, proxied or replayed. The concrete failure is **IDOR**: a valid token with another user's id in the request returning their data. The backend prevents this by scoping the query to the caller. The app cannot prevent it and must not be relied on to.

[NEITHER] Client-supplied identity, amount, price, role or status is never trusted by the backend. When the app sends an amount, the backend re-derives it.

8.4 Platform hardening

- [NEITHER] Sensitive screens block screenshots and screen recording — `FLAG_SECURE` on Android, the equivalent overlay on iOS. Apply to screens showing full account numbers, KYC documents, OTPs or wallet keys.
- [NEITHER] Sensitive content is hidden in the app switcher. The OS snapshots the screen when the app backgrounds, and that snapshot persists.
- [NEITHER] Deep links are validated. A link carries a destination, never an instruction or an authorisation. Verify the target exists and the user may see it before navigating; never act on a parameter without checking it server-side.
- [NEITHER] Nothing sensitive goes to the clipboard without the user explicitly asking, and it is cleared afterwards.

- [NEITHER] **Permissions are requested at the point of use with a plain explanation**, never all at launch. A permission requested with no context is denied, and on iOS a denial is sticky.
- [NEITHER] Biometric or device-credential gating on sensitive actions where the client requires it. Biometrics authorise a local action; they are never a substitute for a server-side check.
- [NEITHER] For apps handling money, **certificate pinning is a per-project decision made by the lead and recorded in the README**, with a documented rotation plan. Pinning without a rotation plan bricks the app when the certificate changes.

8.5 AI assistance

[NEITHER] **Never put secrets, customer data, health data or logs containing personal details into an AI tool's prompt or attachments.** Use approved tools and settings only. Code structure and generic questions are fine; a production payload is not.

9. Dependencies & configuration

- [CI] **Pin the Flutter SDK version** in the repo and commit `pubspec.lock`. **Check:** a CI step that fails when the lock file is missing or out of date.
- [NEITHER] A new package needs, in the PR description: what it does, why Flutter or Dart cannot, its last release date, its licence, and whether it supports both platforms. *Reason:* every package is permanent attack surface and future upgrade work, and a package that drops a platform drops it for us.
- [NEITHER] **Unmaintained packages** — no release in roughly two years — are flagged for replacement rather than adopted. A temporary override or fork needs an owner and a removal ticket.
- [NEITHER] Read upgrade notes, test the affected flows, and remove unused packages in the same PR that removes the feature.
- [NEITHER] **Use the stable release, not the newest.**
- [NEITHER] **Environment values come from `--dart-define` at build time**, read once into a typed config class. Nothing reads a raw define at the call site. **These are configuration, not secrets** — see 8.1.
- [NEITHER] `.env.example` or the equivalent lists every required define with a dummy value.

10. Maintainability & performance

- [NEITHER] **Never start a request, change state, or do expensive work inside `build()`**. `build()` runs on every rebuild — on animation frames, on keyboard open, on parent rebuild. Work belongs in `initState`, an event handler, or a BLOC.
- [LINTER] **Use `const` constructors wherever possible.** A `const` widget is not rebuilt. **Check:** `prefer_const_constructors`, `prefer_const_literals_to_create_immutables`.
- [NEITHER] **Long lists use `ListView.builder` or a sliver, never a `Column` inside a `SingleChildScrollView`**. The second builds every row whether or not it is visible.
- [NEITHER] Stateful list items carry a stable `Key`, not an index. Without one, reordering or filtering attaches the wrong state to the wrong row.
- [NEITHER] **Split a widget when it mixes jobs, nests more than about three levels deep, or repeats business rules.** No fixed line limit — depth and mixed responsibility are the signals that matter in a widget tree.
- [NEITHER] **Measure suspected slowness on a representative device**, not a flagship and not a simulator. Our users are not on the newest phone.
- [NEITHER] Images are sized and cached. A full-resolution photo decoded into a 100px thumbnail is the most common memory problem in a Flutter app.
- [NEITHER] The third repetition of the same logic gets extracted; the second does not.
- [NEITHER] Comments explain **why**, never what. Every README covers setup, required defines, and how to run and test locally.

11. What we do NOT do

- [NEITHER] **No `double` for money, token amounts or interest.** See section 12.
- [LINTER] No `print()`, `debugPrint()` or `dart:developer` logging in app code.
- [LINTER] **No unchecked `!` on data from outside the app** — an API response, a deep link, storage. If it can be null, handle it. **Check:** review item, backed by `avoid_dynamic_calls`.

- [LINTER] No empty catches, and no catch that logs and continues.
- [NEITHER] No requests started inside `build()`.
- [NEITHER] No blind retries of writes. A retried payment is not a harmless retry.
- [NEITHER] No client-only access checks presented as security.
- [NEITHER] No secrets in `SharedPreferences`.
- [NEITHER] **No disabling a lint rule or a test to get CI green.** If a rule is wrong, change it in `analysis_options.yaml` in its own PR with a reason.
- [NEITHER] No adding layers for imagined future needs, and no rewriting unrelated code inside a feature PR.
- [NEITHER] No `setState` in a widget that has a BLoC for the same state. Pick one owner per piece of state.
- [NEITHER] No business logic in a widget. Widgets render; BLoCs and services decide.

12. Money and numeric values

[NEITHER] **double never touches a monetary or token amount.** Not in a model, not in a calculation, not in a `fromJson`.

`0.1 + 0.2` is `0.30000000000000004` in Dart as everywhere else. On a financial platform that produces defects found by clients, not by us.

Kind	Type	Why
Fiat currency	<code>int</code> minor units, received and sent as <code>String</code>	Exact, and matches what our backends send
Token / on-chain	<code>BigInt</code> base units, received and sent as <code>String</code>	Exceeds 64-bit, and matches the chain
Rates and percentages	integer basis points (<code>interestBps: 250 = 2.5%</code>)	No decimal to round

```
// core/money/money.dart

/// Minor units. 1050 == ₹10.50.
extension Money on int {
  int applyBps(int bps) {
    assert(bps >= 0, 'bps must be non-negative');
    return this * bps ~/ 10000; // integer division, truncates
  }
}

String formatMinor(int amountMinor, {int decimals = 2, String symbol = '₹'}) {
  final base = math.pow(10, decimals).toInt();
  final whole = amountMinor ~/ base;
  final frac = (amountMinor % base).toString().padLeft(decimals, '0');
  return '$symbol${_grouped(whole)}.$frac';
}
```

- [NEITHER] **Amounts are parsed from the JSON string, never from a JSON number.** `int.parse(json['principalMinor'] as String)`. A JSON number above 2^{53} loses precision silently, and on Flutter web `int` is a JS number — the loss is real, not theoretical.
- [NEITHER] **Totals, fees and interest are calculated on the backend.** The app displays what it is given. If a screen needs a computed total before submission, it asks the API rather than reproducing the formula — two implementations of one rule will disagree eventually, and the app version cannot be hot-fixed.
- [NEITHER] Formatting happens at the display boundary only, in one helper.

Never:

```
final total = double.parse(a) + double.parse(b); // ✗
final fee = amount * 0.025; // ✗
Text('₹${loan.principal / 100}')
```

13. Widgets & UI

Most of this standard protects us from defects. This section protects us from **rework**. Clients change designs during a project and after launch, and in a mobile app a redesign that touches the wrong layer means a new store release.

13.1 The test

When the design changes, what has to change in the code?

- Theme tokens and widget trees → the architecture is right.
- BLoCs, models, repositories or API calls → the separation was wrong, and we are paying for it in rework.

13.2 Design tokens live in the theme

[NEITHER] Colours, spacing, text styles, radii and elevation are defined once in the theme. A widget never contains a raw value.

Reason: a rebrand, a dark theme, or a white-label build for a second client should touch one file. Hardcoded values turn that into an audit of every screen.

```
// core/theme/app_tokens.dart
@immutable
class AppTokens extends ThemeExtension<AppTokens> {
  const AppTokens({
    required this.surface,
    required this.surfaceMuted,
    required this.textPrimary,
    required this.textMuted,
    required this.border,
    required this.primary,
    required this.danger,
    required this.space1, required this.space2, required this.space4, required this.space8,
    required this.radiusMd,
  });

  final Color surface, surfaceMuted, textPrimary, textMuted, border, primary, danger;
  final double space1, space2, space4, space8;
  final double radiusMd;

  static const light = AppTokens(
    surface: Color(0xFFFFFFFF),
    surfaceMuted: Color(0xFFF4F6F8),
    textPrimary: Color(0xFF13202B),
    textMuted: Color(0xFF5B6670),
    border: Color(0xFFD8DEE4),
    primary: Color(0xFF1A6EA8),
    danger: Color(0xFFC0392B),
    space1: 4, space2: 8, space4: 16, space8: 32,
    radiusMd: 8,
  );

  @override
  AppTokens copyWith({ /* ... */ }) => /* ... */;

  @override
  AppTokens lerp(ThemeExtension<AppTokens>? other, double t) => /* ... */;
}

extension AppTokensX on BuildContext {
  AppTokens get tokens => Theme.of(this).extension<AppTokens>(!);
}
```

```
// ✓ reads from the theme – a rebrand or dark mode is one file
Container(
  padding: EdgeInsets.all(context.tokens.space4),
  decoration: BoxDecoration(
    color: context.tokens.surface,
    borderRadius: BorderRadius.circular(context.tokens.radiusMd),
  ),
)

// ✗ a raw value, and it breaks in dark mode
Container(padding: const EdgeInsets.all(13), color: const Color(0xFFFFFFFF))
```

Check: review item 13. No lint rule distinguishes a token from a literal, which is why this is [NEITHER].

13.3 Three kinds of widget

[NEITHER] Every widget is one of these, and the kind decides what it may contain.

Kind	May use	May do
Screen	BLoC, navigation	Own the screen's state, compose children
Feature widget	Nothing	Know the domain; typed constructor parameters only
Presentational widget	Nothing	Render what it is given. Knows no domain at all

A presentational widget that reads a BLoC or calls a repository **cannot be reused, cannot be tested in isolation, and cannot survive a redesign that moves it to another screen**. That is the expensive mistake in this section.

```
// ✓ presentational – reusable, testable, survives any redesign
class StatusPill extends StatelessWidget {
  const StatusPill({super.key, required this.label, this.tone = Tone.neutral});
  final String label;
  final Tone tone;
  // ...
}

// ✗ not presentational – it reads a BLoC, so it only works on one screen
class StatusPill extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    final state = context.watch<LoanBloc>().state; // ✗
  }
}
```

13.4 Constructor parameters are the contract

[NEITHER] A widget's internal tree may change at any time. **Its constructor parameters may not**. Changing one on a shared widget is a breaking change: add the new parameter, migrate the call sites, then remove the old one.

Presentational widgets take **view data, not domain models**. A widget typed to `Loan` breaks when the API changes; one typed to `label`, `subtitle`, `tone` does not.

13.5 A widget never sets its own outer margin

[NEITHER] A widget owns everything inside its box. **The parent owns where it sits and the space around it**. Use `spacing` on a `Column` / `Row`, or `Padding` in the parent. A widget carrying its own outer margin fights every reuse.

13.6 Accessibility and responsiveness

- [NEITHER] **Text scales with the system setting**. Never a fixed-height container around text that will overflow at 200% scale — test at the largest setting before shipping.
- [NEITHER] Every interactive control has a `Semantics` label; icon-only buttons always.
- [NEITHER] Touch targets at least 48dp.
- [NEITHER] Contrast meets 4.5:1 — checked once against the tokens, not per screen.
- [NEITHER] Layouts handle small phones, tablets, landscape and the keyboard being open. A screen that only works on the developer's device is not done.

13.7 Starting the shared widget library

[NEITHER] **A widget graduates to `core/` or a shared package on its third use, not its first**. Two uses is a coincidence; three is a pattern, and by then you know which parts vary. On graduation it gains a named owner, documented parameters, and the breaking-change rule in 13.4.

14. Mobile platform concerns

14.1 The network is unreliable

[NEITHER] **Poor connectivity is a normal operating condition, not an error case.**

- Every screen that loads data handles the offline case with a message and a retry, not a spinner that never resolves.
- Reads are cached where it makes the app usable offline; the cache shows its age rather than pretending to be live.
- **Writes are never silently retried without an idempotency key.** A retried payment is a second payment.
- A queued write that has not yet been sent is visible to the user as pending, never as done.

14.2 App lifecycle

- [NEITHER] **Sensitive data is cleared or masked when the app backgrounds**, and sensitive screens are hidden from the app switcher snapshot (8.4).
- [NEITHER] The app handles being killed and restored mid-flow. A multi-step form either survives it or restarts cleanly — it does not restore half of itself.
- [NEITHER] Re-authentication on resume after a configured period for apps handling money.
- [NEITHER] Timers, listeners and polling are paused on background and resumed on foreground. A poll that keeps running in the background drains the battery and gets the app uninstalled.

14.3 Both platforms, every time

- [NEITHER] **Any change to a native flow is checked on Android and iOS before the PR is raised.** Permissions, file pickers, notifications, deep links, keyboard behaviour, back navigation and safe areas all differ.
- [NEITHER] Platform differences are handled explicitly, not left to whichever device the developer happened to have.

14.4 Releases

- [CI] **Production builds come from an approved release tag**, with signing, obfuscation (`--obfuscate --split-debug-info`) and debug symbols uploaded to Sentry.
- [NEITHER] Version and build numbers follow one scheme across both platforms, and the release notes say what changed.
- [NEITHER] **A forced-upgrade path exists and is tested** — a minimum supported version the backend can enforce. Without it, a breaking API change strands every user who has not updated, and there is no hot fix for a shipped binary.
- [NEITHER] Release contents, breaking changes and test results are checked before merging the release PR.
- [NEITHER] Staged rollout for a release that touches payments or auth. Watch the crash rate before going to 100%.

Appendix — analysis_options.yaml

Baseline existing violations rather than fixing them all at once.

```
include: package:flutter_lints/flutter.yaml

analyzer:
  language:
    strict-casts: true
    strict-raw-types: true
  errors:
    use_build_context_synchronously: error    # section 3.2 – crashes, not style
    avoid_print: error
    empty_catches: error
    unawaited_futures: error
    prefer_const_constructors: warning
    missing_required_param: error
    missing_return: error
  exclude:
    - "**/*.g.dart"
    - "**/*.freezed.dart"

linter:
  rules:
    - always_declare_return_types
    - avoid_dynamic_calls
    - avoid_empty_else
    - avoid_print
    - avoid_slow_async_io
    - cancel_subscriptions
```

- close_sinks
- empty_catches
- file_names
- library_private_types_in_public_api
- prefer_const_constructors
- prefer_const_literals_to_create_immutables
- prefer_final_fields
- prefer_final_locals
- sized_box_for_whitespace
- sort_child_properties_last
- unawaited_futures
- use_build_context_synchronously
- use_super_parameters

`use_build_context_synchronously`, `cancel_subscriptions` and `close_sinks` are the three rules that catch real crashes rather than style. They are errors, not warnings.